

Interface Textual

1 – Persistência de Objetos em Arquivos e Tratamento de Exceção

Nos três tutoriais anteriores, os atributos dos objetos eram gerados na própria implementação. Neste tutorial, vamos implementar uma aplicação na qual os objetos serão criados a partir da leitura de seus atributos, utilizando uma interface textual. A medida que os atributos forem sendo lidos, os objetos serão criados e armazená-los.

Obviamente não seria nem um pouco agradável informar atributos de vários objetos, através de uma interface textual, e perder o conteúdo desses objetos ao encerrar a aplicação. A solução mais simples para evitar essa situação é persistir (armazenar) os objetos cadastrados em um arquivo antes de encerrar a aplicação, e recuperá-los (trazendo de volta para a memória) no início da aplicação.

Com a utilização da biblioteca `pickle`, é possível de armazenar um conjunto de objetos em arquivo e posteriormente recuperá-los a partir do arquivo em alto nível, com um código bem compacto. Nesta seção, vamos então definir as funções `salvar_arquivo` e `carregar_arquivo` no módulo `persistência_arquivo` do diretório `util`. Essas são as funções mais básicas que iremos utilizar para persistir objetos. As funções de mais alto nível para salvar e recuperar os objetos da aplicação serão definidas oportunamente no módulo do diretório `controle`.

```
import pickle

def salvar_arquivo(nome_arquivo, objetos):
    arquivo = open('../dados/' + nome_arquivo + '.bin', 'wb')
    pickle.dump(objetos, arquivo)
    arquivo.close()

def carregar_arquivo(nome_arquivo):
    try:
        arquivo = open('../dados/' + nome_arquivo + '.bin', 'rb')
        objetos = pickle.load(arquivo)
    except IOError: objetos = None
    return objetos
```

A função `salvar_arquivo` suporta o salvamento de um conjunto de objetos em um arquivo com extensão `bin`. A função `open` está sendo utilizada para criar um arquivo com o nome passado como argumento para o parâmetro `nome_arquivo`. O string `'wb'` passado como argumento para a função `open`, indica que será aberto para escrita (`w` de write : escrever) um arquivo binário (`b` de binary : binário). A função `dump` do módulo `pickle` está sendo utilizada para salvar, em alto nível, o argumento recebido no parâmetro `objetos` (conjunto de objetos) no arquivo criado na variável `arquivo`.

A função `recuperar_arquivo` suporta a recuperação de um conjunto de objetos de um dado arquivo com extensão `bin`. Neste caso, a função `open` está sendo utilizada com o argumento `'rb'` para abrir para leitura (`r` de read : ler) um arquivo binário. No entanto, a não existência desse arquivo poderia resultar na geração de uma exceção. Para evitar que isso aconteça, os comandos de leitura e carregamento do arquivo são encapsulados por um tratador de exceção.

Exceção é uma situação cuja ocorrência, na execução de uma aplicação, pode encerrar de forma mal sucedida a execução da aplicação. Algumas operações tem um potencial para causar uma exceção, como por exemplo: (a) divisão por uma variável, poderá causar uma exceção, se foi atribuído previamente o valor zero a essa variável; (b) acesso a um arquivo inexistente. Nestes casos, as operações que tem o potencial de causar uma exceção devem ser confinadas em um bloco de comandos interno à palavra reservada `try` (tentar), o que sinaliza que a implementação tentará executar comandos potencialmente causadores de exceção. A palavra reservada `except` (exceto) indica que se ocorrer o tipo de exceção, cuja classe de exceção é especificada (que neste caso é `IOError` : erro de entrada e saída), o bloco de comandos interno ao `except` será executado, em vez de causar o encerramento mal sucedido da aplicação.

2 – Interface Textual

Agora vamos ilustrar as funções que permitirão que você leia os dados dos objetos que serão utilizados para criar os objetos das entidades do projeto de referência. Todas as funções que serão ilustradas nesta seção pertencem ao módulo `interface_textual` do diretório `interfaces`.

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos, imprimir_objetos_associação_filtros
from util.data import converte_str_para_data
from entidades.montadora import Montadora, inserir_montadora, get_montadoras
from entidades.agência_publicidade import AgênciaPublicidade, inserir_agência_publicidade, get_agências_publicidade
from entidades.veículo import Carro, Caminhão
from entidades.lançamento import Lançamento, filtrar_lançamentos, get_lançamentos, criar_lançamento, inserir_lançamento
```

Ao inserir o código no PyCharm, você poderá fazê-lo de forma bottom-up (do nível mais detalhado para o mais alto), para que o PyCharm não fique assinalando a falta de funções ainda não definidas, mas para facilitar o entendimento, vamos explicar de forma top-down (do nível mais alto para o nível mais detalhado).

O que o menu da sua aplicação precisa oferecer como opções: (a) criação e impressão dos objetos das entidades `AgênciaPublicidade`, `Montadora` e `Veículo` (serão criados objetos das subclasses `Carro` e `Caminhão`) e `Lançamento`; e (b) leitura de valores de filtros para seleção de lançamentos e impressão dos lançamentos selecionadas. A função `loop_opções_execução`, que controla o loop de opções disponíveis para a execução da aplicação, é ilustrada na próxima página. Essa função utiliza a função `ler_str`, que será ilustrada posteriormente, dado que estamos utilizando um abordagem top-down.

Para minimizar a digitação na escolha de opções é utilizada apenas um letra maiúscula para caracterizar a opção. Para sair do loop mais externo de opções, ou para retornar do loop mais interno para o mais externo é utilizada a tecla `ENTER` para agilizar. Ao selecionar a opção cadastrar você executará um loop de cadastros da entidade selecionada e ao sair do loop, os objetos dessa entidade (criados nas execuções anteriores acrescidos dos criados na execução em andamento) serão mostradas da tela. Você também pode optar por executar o loop de seleção de reservas, e cada conjunto de filtros escolhidos, as reservas selecionadas serão mostradas na tela. Adicionalmente, você tem a opção de selecionar a impressão de todos os objetos cadastrados das entidades da aplicação.

```
def loop_opções_execução():
    sair_loop = False
    cabeçalho_agência_publicidade = '\nAgências de Publicidades : nome - país de origem - ranking de avaliação'
    cabeçalho_montadora_veículos = ('\nMontadoras : nome - país de origem - sede da maior fábrica no Brasil'
    + '\n - Veículos : marca modelo - alimentação - potência - carro:[tipo - volume_carga] | caminhão:[tração - capacidade de carga]'
    + ' - importado')
    cabeçalho_lançamento = ('\nLançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade'
    + ' - data do lançamento')

    while not sair_loop:
        print()
        operação = ler_str('Opções [C: Cadastrar / I: Imprimir / S: Selecionar / T: imprimir Todos / <ENTER>: Parar]', retornar=True)
        if operação == None: break
        elif operação in ('C', 'I'):
            opção_conteúdo = ler_str('A: Agências de Publicidade / M: Montadoras / L: Lançamentos / <ENTER>: retornar', retornar=True)
            if opção_conteúdo == None: pass
            elif opção_conteúdo == 'A':
                if operação == 'C': loop_leitura_agências_publicidade()
                imprimir_objetos(cabeçalho_agência_publicidade, get_agências_publicidade().values())
            elif opção_conteúdo in 'M':
                if operação == 'C': loop_leitura_montadoras()
                imprimir_montadoras_veículos(cabeçalho_montadora_veículos)
            elif opção_conteúdo == 'L':
                if operação == 'C': loop_leitura_lançamentos()
                imprimir_objetos(cabeçalho_lançamento, get_lançamentos())
        elif operação == 'S': loop_seleção_lançamentos()
        elif operação == 'T':
            imprimir_objetos(cabeçalho_agência_publicidade, get_agências_publicidade().values())
            imprimir_montadoras_veículos(cabeçalho_montadora_veículos)
            imprimir_objetos(cabeçalho_lançamento, get_lançamentos())
```

A função `imprimir_montadoras` de veículos, utilizada para imprimir as montadoras, e os veículos de cada montadora, é ilustrada abaixo.

```
def imprimir_montadoras_veículos(cabeçalho_montadora_veículos):
    print(cabeçalho_montadora_veículos)
    for índice, montadora in enumerate(get_montadoras().values()):
        imprimir_objeto(indíce=índice, objeto_str=str(montadora))
        imprimir_objetos_internos(montadora.veículos.values())
```

Vamos ilustrar as funções que controlam o loop de criação de objetos da entidades e a seleção de lançamentos. Adicionalmente, vamos ilustrar a função `ler_sair_loop`, que apesar ser uma função bem básica (baixo nível) é chamada somente pelas quatro funções de controle de loop.

```
def loop_leitura_agências_publicidade():
    sair_loop = False
    print('--- Leitura de Dados das Agências de Publicidade ---')
    while not sair_loop:
        agência_publicidade = ler_agência_publicidade()
        if agência_publicidade is not None: inserir_agência_publicidade(agência_publicidade)
        else: print(' - ERRO : na leitura do agência_publicidade')
        sair_loop = ler_sair_loop('cadastro de agências_publicidade')

def loop_leitura_montadoras():
    sair_loop = False
    print('--- Leitura de Dados das Montadoras ---')
    while not sair_loop:
        montadora = ler_montadora()
        if montadora is not None:
            inserir_montadora(montadora)
            loop_leitura_veículos_montadora(montadora)
        else: print(' - ERRO : na leitura da montadora')
        sair_loop = ler_sair_loop('cadastro de montadoras')
```

```
def loop_leitura_veiculos_montadora(montadora):
    sair_loop = False
    print('--- Leitura de Dados dos Veículos da Montadora : ' + montadora.nome + ' ---')
    while not sair_loop:
        veiculo = ler_veiculo()
        if veiculo is not None: montadora.inserir_veiculo(veiculo)
        else: print(' - ERRO : na leitura de veículo')
        sair_loop = ler_sair_loop('cadastro de veículos da montadora')

def loop_leitura_lancamentos():
    sair_loop = False
    print('--- Leitura de Dados de Lançamentos ---')
    while not sair_loop:
        lancamento = ler_lancamento()
        if lancamento is not None: inserir_lancamento(lancamento)
        else: print(' - ERRO : na leitura de lançamento')
        sair_loop = ler_sair_loop('cadastro de lançamento')

def ler_sair_loop(loop):
    try:
        sair = input('-- sair do loop de ' + loop + ' [S]: ')
        if sair == 'S': return True
    except IOError: pass
    return False
```

Nessas três funções de controle de loop de leitura dos dados das entidades da aplicação, é chamado o método de leitura do respectivo objeto da entidade. O retorno de um objeto não nulo, indicará que a leitura de todos os dados da entidade foram lidos com sucesso e foi criado um objeto para a entidade.

Dado que o usuário poderá criar vários objetos em sequência, a função `ler_sair_loop`, foi criada para permitir a continuidade no loop simplesmente por digitar a tecla `ENTER`. Para sair do loop, será necessário digitar a letra maiúscula `'S'`.

Na função `loop_seleção_lancamentos`, após cada leitura de conjunto de filtros, os lançamentos selecionados são impressos.

```
def loop_seleção_lancamentos():
    sair_loop = False
    print('--- Seleção de Lançamentos ---')
    while not sair_loop:
        filtros, lancamentos_selecionados = selecionar_lancamentos()
        if filtros is not None:
            cabeçalho =
                ('Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento'
                 + '\n -- ranking de avaliação da agência de publicidade - país de origem da montadora - potência do veículo'
                 + '\n - carro:[tipo] | caminho:[capacidade de carga]')
            imprimir_objetos_associação_filtros(cabeçalho, lancamentos_selecionados, filtros)
        sair_loop = ler_sair_loop('seleção de lançamentos')
```

Na sequência, vamos ilustrar cada uma das funções de leitura de objetos e a função de seleção de lançamentos.

```
def ler_agência_publicidade():
    nome = ler_str('nome da agência_publicidade')
    if nome == None: return None
    país_origem = ler_str('país de origem da agência_publicidade')
    if país_origem == None: return None
    ranking_avaliação = ler_int_positivo('ranking de avaliação da agência_publicidade')
    if ranking_avaliação == None: return None
    return AgênciaPublicidade(nome, país_origem, ranking_avaliação)
```

```
def ler_montadora():
    nome = ler_str('nome da montadora')
    if nome == None: return None
    país_origem = ler_str('país de origem da montadora')
    if país_origem == None: return None
    sede_maior_fábrica_brasil = ler_str('sede da maior fábrica do brasil da montadora [Cidade - UF]')
    if sede_maior_fábrica_brasil == None: return None
    return Montadora(nome, país_origem, sede_maior_fábrica_brasil)
```

Na função `ler_agência_publicidade`, são lidos os atributos da classe `AgênciaPublicidade` e retornado o objeto criado. Caso falhe a leitura algum dos atributo (função utilizada para ler o atributo retorna `None`), o objeto não será criado e será retornado `None`. Estes testes de falha de leitura de atributos são necessários em todas as funções de criação de objetos, porque somente com todos os atributos lidos com sucesso, será possível criar o objeto. Na função `ler_montadora`, o procedimento é semelhante.

```
def ler_veículo():
    marca_modelo = ler_str('marca modelo do veículo')
    if marca_modelo == None: return None
    alimentação = ler_alimentação_veículo('alimentação do veículo')
    if alimentação == None: return None
    potência = ler_int_positivo('potência do veículo')
    if potência == None: return None
    importado = ler_bool('veículo importado')
    if importado == None: return None
    espécie_veículo = ler_str('espécie do veículo [Cr=Carro / Cm=Caminhão]')
    if espécie_veículo == 'Cr':
        tipo = ler_tipo_carro()
        if tipo == None: return None
        volume_carga = ler_int_positivo('volume de carga do carro (litros)')
        if volume_carga == None: return None
        return Carro(marca_modelo, alimentação, potência, importado, tipo, volume_carga)
    if espécie_veículo == 'Cm':
        tração = ler_tração_caminhão()
        if tração == None: return None
        capacidade_carga = ler_int_positivo('capacidade de carga do caminhão')
        if capacidade_carga == None: return None
        return Caminhão(marca_modelo, alimentação, potência, importado, tração, capacidade_carga)
    else: return None
```

Na função `ler_veículo`, inicialmente são lidos os atributos comuns herdados da classe `Veículo`. A seguir, é solicitado o tipo de veículo a ser criado e lidos os demais atributos da subclasse escolhida e, então, criado e retornado o objeto da respectiva subclasse.

```
def ler_lançamento():
    nome_montadora = ler_str('nome da montadora')
    if nome_montadora == None: return None
    marca_modelo_veículo = ler_str('marca modelo do veículo')
    if marca_modelo_veículo == None: return None
    nome_agência_publicidade = ler_str('nome da agência de publicidade')
    if nome_agência_publicidade == None: return None
    data_lançamento = ler_data('data do lançamento')
    if data_lançamento is None: return None
    return criar_lançamento(nome_montadora, marca_modelo_veículo, nome_agência_publicidade, data_lançamento)
```

Na função `ler_lançamento`, o objeto da classe `Lançamento` é criado a partir de três objetos das classes `Montadora`, `Veículo` e `AgênciaPublicidade` previamente criados. Para obter cada um desses objetos, é lida a respectiva chave e obtido o objeto no dicionário que o armazena. Caso, algum dos objetos não exista, a criação do objeto da classe `Lançamento` é mal sucedida.

Na função `selecionar_lançamentos` serão feitos testes para criar o string representando os valores fornecidos para os filtros. Com esses valores é chamada a função `filtrar_lançamentos` do módulo `lançamento`, e retornados o string dos filtros e os lançamentos selecionados.

```
def selecionar_lançamentos():
    filtros = "\nFiltros --"
    data_mínima_lançamento = ler_data('data mínima do lançamento', filtro=True)
    if data_mínima_lançamento is not None: filtros += 'data mínima do lançamento: ' + str(data_mínima_lançamento)
    potência_mínima_veículo = ler_int_positivo('potência mínima do veículo', filtro=True)
    if potência_mínima_veículo is not None: filtros += ' - potência mínima do veículo: ' + str(potência_mínima_veículo)
    tipo_carro = ler_tipo_carro(filtro=True)
    if tipo_carro is not None: filtros += '\n - tipo do carro: ' + tipo_carro
    capacidade_mínima_carga_caminhão = ler_int_positivo('capacidade mínima de carga do caminhão', filtro=True)
    if capacidade_mínima_carga_caminhão is not None: filtros += ' - capacidade mínima de carga do caminhão: ' + str(capacidade_mínima_carga_caminhão)
    ranking_máximo_avaliação_agência_publicidade = ler_int_positivo('ranking máximo de avaliação da agência de publicidade', filtro=True)
    if ranking_máximo_avaliação_agência_publicidade is not None:
        filtros += ' - ranking máximo de avaliação da agência de publicidade: ' + str(ranking_máximo_avaliação_agência_publicidade)
    país_origem_montadora = ler_str('país de origem da montadora', filtro=True)
    if país_origem_montadora is not None: filtros += '\n - país de origem da montadora: ' + país_origem_montadora
    lançamentos_selecionados = filtrar_lançamentos(data_mínima_lançamento, potência_mínima_veículo, tipo_carro, capacidade_mínima_carga_caminhão, ranking_máximo_avaliação_agência_publicidade, país_origem_montadora)
    return filtros, lançamentos_selecionados
```

Para complementar a implementação necessária para filtragem de lançamentos é necessário implementar a função `filtrar_lançamentos` no módulo `lançamento`, que receberá os valores dos filtros, lidos na função `selecionar_reservas` do módulo `interface_textual`, e retorna os lançamentos filtrados.

```
def filtrar_lançamentos(data_mínima_lançamento, potência_mínima_veículo, tipo_carro, capacidade_mínima_carga_caminhão, ranking_máximo_avaliação_agência_publicidade, país_origem_montadora):
    lançamentos_selecionados = []
    for lançamento in lançamentos:
        if data_mínima_lançamento is not None and lançamento.data < data_mínima_lançamento: continue
        if potência_mínima_veículo is not None and lançamento.veículo.potência < potência_mínima_veículo: continue
        if isinstance(lançamento.veículo, Carro):
            if tipo_carro is not None and lançamento.veículo.tipo != tipo_carro: continue
        elif isinstance(lançamento.veículo, Caminhão):
            if capacidade_mínima_carga_caminhão is not None and lançamento.veículo.capacidade_carga < capacidade_mínima_carga_caminhão: continue
        if (ranking_máximo_avaliação_agência_publicidade is not None
            and lançamento.agência_publicidade.ranking_avaliação > ranking_máximo_avaliação_agência_publicidade): continue
        if país_origem_montadora is not None and lançamento.montadora.país_origem != país_origem_montadora: continue
        lançamentos_selecionados.append(lançamento)
    return lançamentos_selecionados
```

Finalmente, vamos ilustrar as classes de mais baixo nível do módulo `interface_textual`, utilizadas para a leitura dos atributos, chaves ou filtros utilizados nas funções de mais alto nível.

Na função `ler_str`, caso seja informado algum caracter inválido, a leitura do string é mal sucedida e a função retorna `None`. A digitação somente da tecla `ENTER` é aceitável somente para filtros, ou caso o usuário queira retornar para o loop de nível superior, ou finalizar a execução da aplicação. Caso o usuário digite somente `ENTER` na leitura de um atributo, a leitura será considerada mal sucedida.

```
def ler_str(dado, filtro=False, retornar=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and (filtro or retornar): return None
        if len(string) > 0: return string
    except IOError: pass
    print('Erro na leitura do dado: ' + dado)
    return None
```

Na função `ler_int_positivo`, queremos ler um string que possa ser convertido para um inteiro positivo. Caso a conversão do string para inteiro falhe, ocorrerá a exceção `ValueError`, que caracteriza a falha na conversão. A função `ler_float_positivo`, é equivalente para valores decimais positivos.

```
def ler_int_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and filtro: return None
        int_positivo = int(string)
        if int_positivo > 0: return int_positivo
    except ValueError: pass
    print('Erro na leitura/conversão do inteiro positivo: ' + dado)
    return None

def ler_float_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and filtro: return None
        float_positivo = float(string)
        if float_positivo > 0.0: return float_positivo
    except ValueError: pass
    print('Erro na leitura/conversão do flutuante positivo: ' + dado)
    return None
```

Na função `ler_bool`, as opções de 'S' e 'N' são interpretadas respectivamente como `True` e `False`.

```
def ler_bool(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [S/N]: ')
        if len(string) == 0 and filtro: return None
        if string == 'S': return True
        elif string == 'N': return False
    except ValueError: pass
    print('Erro na leitura do booleano: ' + dado)
    return None
```

Para ler o string de uma data e convertê-lo para um objeto data, utilizaremos a função `ler_data`.

```
def ler_data(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [dd/mm/aaaa]: ')
        if len(string) == 0 and filtro: return None
        data = converte_str_para_data(string)
        if data is not None: return data
    except IOError: pass
    print('Erro na leitura da data: ' + dado)
    return None
```

A função `converte_str_para_data`, definida no módulo `data` do diretório `gerais`, utiliza uma expressão regular para consistir a sintaxe da data informada pelo usuário.

```
def converte_str_para_data(data_str):
    if not match('(0?[1-9])[12][0-9][301])/(0?[1-9][1012])\d{4}', data_str): return None
    dia, mês, ano = data_str.split('/')
    return Data(int(dia), int(mês), int(ano))
```


As funções ilustradas até o momento são bem genéricas, mas para lermos um string que faça parte de uma tupla específica (um enumerado), precisaremos definir uma função específica. As funções específicas para leitura de enumerados, no projeto de referência, são: [ler_alimentação_veículo](#), [ler_tipo_carro](#) e [ler_tração_caminhão](#).

```
def ler_alimentação_veículo(filtro=False):
    try:
        string = input('- alimentação do veículo [F=flex / D=diesel / E=elétrico]: ')
        if len(string) == 0 and filtro: return None
        if string == 'F': return 'flex'
        if string == 'D': return 'diesel'
        if string == 'E': return 'elétrico'
    except IOError: pass
    print('Erro na leitura da alimentação do veículo')
    return None

def ler_tipo_carro(filtro=False):
    try:
        string = input('- tipo do carro [P=passeio / U=utilitário / C=caminhonete]: ')
        if len(string) == 0 and filtro: return None
        if string == 'P': return 'passeio'
        if string == 'U': return 'utilitário'
        if string == 'C': return 'caminhonete'
    except IOError: pass
    print('Erro na leitura do tipo do carro')
    return None

def ler_tração_caminhão(filtro=False):
    try:
        string = input('- tração do caminhão [A=6x4 / B=6x2 / C=4x4]: ')
        if len(string) == 0 and filtro: return None
        if string == 'A': return '6x4'
        if string == 'B': return '6x2'
        if string == 'C': return '4x4'
    except IOError: pass
    print('Erro na leitura da tração do caminhão')
    return None
```

3 – Controle do Projeto

Para finalizar, vamos ilustrar o módulo [divulgação_lançamento_veículos](#) do diretório [controle](#).

```
from util.persistência_arquivo import carregar_arquivo, salvar_arquivo
from entidades.agência_publicidade import get_agências_publicidade, set_agências_publicidade
from entidades.montadora import get_montadoras, set_montadoras
from entidades.lançamento import get_lançamentos, set_lançamentos
from interfaces.interface_textual import loop_opções_execução
nome_arquivo = 'lançamento_veículos'

def salvar_aplicação():
    lançamento_veículos = []
    lançamento_veículos.append(get_agências_publicidade())
    lançamento_veículos.append(get_montadoras())
    lançamento_veículos.append(get_lançamentos())
    salvar_arquivo(nome_arquivo, objetos=lançamento_veículos)

def recuperar_aplicação():
    lançamento_veículos = carregar_arquivo(nome_arquivo)
    if lançamento_veículos is not None:
        set_agências_publicidade(lançamento_veículos[0])
        set_montadoras(lançamento_veículos[1])
        set_lançamentos(lançamento_veículos[2])

if __name__ == '__main__':
    recuperar_aplicação()
    loop_opções_execução()
    salvar_aplicação()
```


O bloco principal da aplicação, inicia executando a função `recuperar_aplicação`, que carrega do arquivo a lista `lançamento_veículos`, que contém os conjuntos de objetos: `agências_publicidade`, `montadoras` (e seus veículos) e `lançamentos`. Esses conjuntos são salvos nos conjuntos de objetos dos módulos `agência_publicidade`, `montadora` e `lançamento`, utilizando as funções importadas desses módulos: `set_consultentes`, `set_obras` e `set_reservas`. A execução da função `recuperar_aplicação`, no início da aplicação, garante que todos os objetos cadastrados previamente sejam recuperados para a memória da aplicação.

```
def set_agências_publicidade(agências_publicidade1):
    global agências_publicidade
    agências_publicidade = agências_publicidade1
```

```
def set_montadoras(montadoras1):
    global montadoras
    montadoras = montadoras1
```

```
def set_lançamentos(lançamentos1):
    global lançamentos
    lançamentos = lançamentos1
```

A seguir é executada a função `loop_opções_execução`, importada do módulo `interface_textual`, que controla o loop de criação e impressão de objetos de consultentes, obras e reservas, bem como a seleção de reservas.

Antes de finalizar, é chamada a função `salvar_aplicação`, que cria a lista `lançamento_veículos`, apendando os conjuntos de objetos `agências_publicidade`, `montadoras` (e seus veículos) e `lançamentos`, e salvando a lista em um arquivo. A leitura dos conjuntos de objetos é realizada com a utilização das funções `get_agências_publicidade`, `get_montadoras` e `get_lançamentos`, importadas dos módulos: `agência_publicidade`, `montadora` e `lançamento`. A execução da função `salvar_aplicação`, ao final da aplicação, garante que todos os objetos cadastrados até o presente momento sejam salvos em arquivo.

Na execução do projeto, várias opções são possíveis. Escolhemos ilustrar duas das opções disponíveis: (a) a impressão de todos os objetos cadastrados; e (b) a filtragem de lançamentos com todos os valores de filtros. Vamos iniciar, ilustrando a impressão de todos os objetos.

```
- Opções [C: Cadastrar / I: Imprimir / S: Selecionar / T: imprimir Todos / <ENTER>: Parar] : T
```

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | BETC Havas | França          | 1 |
2 - | Galeria   | Brasil          | 2 |
3 - | Artplan   | Brasil          | 3 |
4 - | McCann    | Estados Unidos | 4 |
5 - | Africa    | Estados Unidos | 5 |
```

Linguagem de Programação I - Tutorial 5 - 10/10

```
Montadoras : nome - país de origem - sede da maior fábrica no Brasil
- Veículos : marca modelo - alimentação - potência - carro:[tipo - volume_carga] | caminhão:[tração - capacidade de carga] - importado
1 - | Hyundai | Coréia do Sul | Piracicaba - SP |
- | Hyundai HB20 | flex | 80 cv | passeio | 475 l |
2 - | GM | Estados Unidos | São Caetano - SP |
- | Chevrolet Onix Plus | flex | 116 cv | passeio | 500 l |
3 - | Fiat | Itália | Betim - MG |
- | Fiat Strada | diesel | 130 cv | caminhonete | 1354 l |
- | Fiat Toro | diesel | 185 cv | caminhonete | 1225 l |
4 - | BYD | China | Camaçari - BA |
- | BYD Dolphin | elétrico | 95 cv | passeio | 250 l | importado |
5 - | Volvo | Suécia | exterior |
- | Volvo XC40 | elétrico | 231 cv | passeio | 419 l | importado |
6 - | Volvo Caminhões | Suécia | Curitiba - PR |
- | Volvo FH 540 | diesel | 540 cv | 6x4 | 80000 kg |
- | Volvo FH 460 | diesel | 460 cv | 6x2 | 38000 kg |
7 - | Volkswagen Caminhões | Alemanha | Resende - RJ |
- | VW Delivery 11180 | diesel | 175 cv | 4x4 | 7500 kg |
8 - | DAF | Holanda | Ponta Grossa - PR |
- | DAF XF 530 | diesel | 530 cv | 6x4 | 74000 kg |
```

```
Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento
1 - | Hyundai | Hyundai HB20 | Africa | 01/09/2012 |
2 - | GM | Chevrolet Onix Plus | McCann | 12/09/2019 |
3 - | Fiat | Fiat Strada | Galeria | 01/09/1998 |
4 - | Fiat | Fiat Toro | Galeria | 01/09/2016 |
5 - | BYD | BYD Dolphin | Artplan | 28/06/2023 |
6 - | Volvo | Volvo XC40 | BETC Havas | 15/04/2018 |
7 - | Volvo Caminhões | Volvo FH 540 | BETC Havas | 01/03/2014 |
8 - | Volkswagen Caminhões | VW Delivery 11180 | Artplan | 23/01/2018 |
9 - | DAF | DAF XF 530 | Artplan | 18/08/2020 |
10 - | Volvo Caminhões | Volvo FH 460 | BETC Havas | 17/04/2014 |
```

A seguir, vamos ilustrar a a filtragem de lançamentos com todos os valores de filtros.

```
- Opções [C: Cadastrar / I: Imprimir / S: Selecionar / T: imprimir Todos / <ENTER>: Parar] : S
--- Seleção de Lançamentos ---
- data mínima do lançamento [dd/mm/aaaa]: 01/01/2012
- potência mínima do veículo : 90
- tipo do carro [P=passeio / U=utilitário / C=caminhonete]: P
- capacidade mínima de carga do caminhão : 20000
- ranking máximo de avaliação da agência de publicidade : 3
- país de origem da montadora : Suécia

Filtros -- data mínima do lançamento: 01/01/2012 - potência mínima do veículo: 90
- tipo do carro: passeio - capacidade mínima de carga do caminhão: 20000 - ranking máximo de avaliação da agência de publicidade: 3
- país de origem da montadora: Suécia
Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento
-- ranking de avaliação da agência de publicidade - país de origem da montadora - potência do veículo
- carro:[tipo] | caminhão:[capacidade de carga]
1 - | Volvo | Volvo XC40 | BETC Havas | 15/04/2018 | 1 | Suécia | 231 cv | passeio |
2 - | Volvo Caminhões | Volvo FH 540 | BETC Havas | 01/03/2014 | 1 | Suécia | 540 cv | 80000 kg |
3 - | Volvo Caminhões | Volvo FH 460 | BETC Havas | 17/04/2014 | 1 | Suécia | 460 cv | 38000 kg |
-- sair do loop de seleção de lançamentos [S]: S
```

Neste ponto, finalizamos o Tutorial 4 com a ilustração completa da Etapa 4 do projeto de referência. Ao finalizarmos a Etapa 4, o projeto de referência está completo.